

LAPORAN PRATIKUM PEKAN 8 STRUKTUR DATA

“Sort Algorithm II”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “Sort Algorithm II”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 25 Mei 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 30 Mei 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Pratikum.....	1
BAB II PEMBAHASAN	2
2.1 ShellSort.....	2
2.2 QuickSort	5
2.3 MergeSort.....	7
2.4 BubbleSortGUI	9
BAB III KESIMPULAN	16
1.1 Kesimpulan	16
1.2 Saran.....	16
BAB IV DAFTAR PUSTAKA	17
Lampiran Tugas	19
1. Deskripsi Tugas.....	19
2. Aturan Penamaan	19
3. Struktur Program.....	19
6. Kriteria Penilaian	20
7. Deadline	21
Java (ADT) Class · lagu_2511531009	22
Java (Main/Driver) Class · sorting_2511531009	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Seperti pertemuan kemarin, kita mengetahui bahwa mengolah data dalam jumlah memerlukan efisiensi yang tinggi, dan untuk meningkatkan efisiensi tersebut kita dapat melakukan pengurutan (sorting) data.

Sorting data juga berguna untuk merapikan data yang telah kita input baik secara *unintentional* maupun hanya sekedar sebagai fitur implisit saja dalam sebuah program yang kita kembangkan.

Pada pertemuan ini kita mempelajari cara kerja algoritma sorting menggunakan metode Shell Sort, Quick Sort, MergeSort, terutama dalam mengurutkan elemen-elemen yang ada pada sebuah array dalam bahasa pemrograman Java. Tidak hanya itu, kita juga akan mengaplikasikan Bubble Sort dengan memanfaatkan JSwing pada library WindowBuilder untuk menghasilkan GUI (Graphical User Interface) yang dapat menunjukkan langkah-langkah pengerjaan algoritma Bubble Sort secara interaktif.

1.2 Tujuan

1. Memahami algoritma Shell Sort dalam bahasa pemrograman Java.
2. Memahami algoritma Quick Sort dalam bahasa pemrograman Java.
3. Memahami algoritma Merge Sort dalam bahasa pemrograman Java.
4. Memahami penerapan lagkah kerja algoritma Bubble Sort dalam bahasa pemrograman Java menggunakan Library WindowBuilder.

1.3 Manfaat Pratikum

1. Mampu memprogram algoritma Shell Sort dalam bahasa pemrograman Java
2. Mampu memprogram algoritma Quick Sort dalam bahasa pemrograman Java
3. Mampu memprogram algoritma Merge Sort dalam bahasa pemrograman Java
4. Mampu mengaplikasikan Insertion Sort dalam sebuah GUI menggunakan library WindowBuilder

BAB II PEMBAHASAN

2.1 ShellSort

Shell Sort merupakan pengembangan dari algoritma Insertion Sort yang dirancang untuk mengatasi kelemahan utamanya, yaitu banyaknya pergeseran elemen yang diperlukan ketika mengurutkan data dalam jumlah yang besar. Shell Sort bekerja dengan cara memulai dengan nilai gap terbesar terlebih dahulu. Nilai gap diperoleh dari $\frac{arr.length}{2}$ dimana *arr.length* merupakan jumlah elemen yang ada di dalam array yang digunakan untuk proses pengurutan.

Pengurutan juga dimulai dari elemen terjauh yang dapat digapai dengan batasan gap yang dimiliki. Pertukaran hanya terjadi ketika $arr[index] > arr[index+gap]$. Setiap kali program mencapai $gap \vee arr.length$ maka gap akan di-divide lagi dengan dua kembali ($gap = \frac{gap}{2}$), hal ini akan terus berulang hingga $gap = 1$.

Proses pengurutan juga dilakukan secara swapping antara nilai *index* dengan nilai (*index + gap*)-nya dengan memanfaatkan variabel temporal. Setelah di swap, nilai *index* yang telah di-swap juga di periksa kembali apakah nilai tersebut lebih besar dibandingkan dengan nilai sebelumnya

```

1 package pekan8_2511531009;
2
3 public class ShellSort_2511531009 {
4
5     public static void ShellSort_1009(int[] A_1009) {
6         int n_1009 = A_1009.length;
7         int gap_1009 = n_1009/2;
8         while (gap_1009 > 0) {
9             for (int i_1009 = gap_1009; i_1009 < n_1009; i_1009++) {
10                 int temp_1009 = A_1009[i_1009];
11                 int j_1009 = i_1009;
12                 while (j_1009 >= gap_1009 && A_1009[j_1009 - gap_1009] > temp_1009) {
13                     A_1009[j_1009] = A_1009[j_1009 - gap_1009];
14                     j_1009 = j_1009 - gap_1009;
15                 }
16                 A_1009[j_1009] = temp_1009;
17             }
18             gap_1009 = gap_1009 / 2;
19         }
20     }
21
22     public static void main(String[] args) {
23         int[] data_1009 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
24
25         System.out.print("Sebelum : ");
26         printArray_1009(data_1009);
27
28         ShellSort_1009(data_1009);
29
30         System.out.print("Sesudah (Shell Sort) : ");
31         printArray_1009(data_1009);
32     }
33
34     public static void printArray_1009(int[] arr_1009) {
35         for (int i_1009 : arr_1009)
36             System.out.print(i_1009 + " ");
37         System.out.println();
38     }
39 }

```

Program yang simpel dengan menggunakan algoritma Shell Sort hanya cukup menggunakan satu method untuk Shell Sort dan satu method untuk mencetak array untuk visualisasi.

Program menggunakan tiga loop, dengan dua diantaranya merupakan while-loop dan satu lainnya menggunakan for-loop

Output :

```

<terminated> ShellSort_2511531009 [Java Application] C:\Users\hamdi\p2\pool\plug
Sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort) : 1 2 3 4 5 6 7 8 9 10

```

Shell Sort Memiliki range kompleksitas waktu sebagai berikut :

Aspek	Kompleksitas	Keterangan
<i>Best Case</i>	$O(n \log n)$	Ketika array telah terurut sempurna
<i>Average Case</i>	$[O(n^{1.25}), O(n^{1.5})]$	Bergantung kepada sekuens gap yang digunakan
<i>Worst Case</i>	$O(n^2)$	Terjadi ketika algoritma membutuhkan

		beberapa gap untuk mengurutkan array
--	--	--------------------------------------

Kompleksitas ruang yang dimiliki oleh algoritma ini adalah $O(1)$ karena merupakan salah satu algoritma yang melakukan pengurutan dataset di tempatnya langsung, dan tidak memerlukan memori tambahan.

Dengan menggunakan algoritma Shell Sort terdapat beberapa keuntungan. Keuntungan dari menggunakan Shell Sort ialah :

- Efisien untuk penggunaan array dengan ukuran sedang
- Hanya memerlukan ukuran memori yang kecil, tanpa array tambahan
- Merupakan bentuk yang lebih *advanced* dari insertion sort, karena memiliki fitur gap yang mempercepat algoritma
- Sederhana untuk di implementasikan
- Dapat berfungsi dengan lancar jika array telah sebagian terurut

Walaupun demikian, kekurangan Shell Sort ialah :

- Tidak cukup stabil
- Performa bergantung pada sekuens gap
- Tidak menjamin array kompleksitas yang terburuk
- Tidak cocok untuk dataset yang besar

Dengan mengetahui keuntungan dan kekurangan, kita dapat memahami kapan untuk menggunakan algoritma Shell Sort :

- Ketika kita memiliki memori yang terbatas
- Ketika data yang kita miliki telah sedikit terurut.
- Ketika membutuhkan algoritma yang efisien
- Ketika ketidakstabilan bukan merupakan masalah
- Ketika jika kita ingin meningkatkan efisiensi algoritma Insertion Sort

2.2 QuickSort

Quick Sort merupakan algoritma pengurutan yang bekerja berdasarkan strategi divide and conquer. Algoritma ini memilih satu elemen sebagai pivot, yang digunakan sebagai acuan untuk membagi array menjadi dua bagian: elemen yang lebih kecil dari pivot dipindahkan ke sisi kiri, dan elemen yang lebih besar dipindahkan ke sisi kanan.

Proses pemindahan elemen dilakukan secara swapping menggunakan variabel temporal. Setelah partisi selesai, pivot berada pada posisi akhirnya yang benar di dalam array. Quick Sort kemudian dipanggil secara rekursif pada sub-array kiri dan kanan hingga setiap sub-array hanya memiliki satu elemen, yang menjadi kondisi dasar (base case) dari rekursi.

```
1 package pekan8_2511531009;
2
3 public class QuickSort_2511531009 {
4
5     static void swap_1009(int[] arr_1009, int i_1009, int j_1009) {
6         int temp_1009 = arr_1009[i_1009];
7         arr_1009[i_1009] = arr_1009[j_1009];
8         arr_1009[j_1009] = temp_1009;
9     }
10
11
12     // additional method : regulate pivot using Median-Of-Three
13     static void medianOfThree(int[] arr_1009, int low_1009, int high_1009) {
14         int mid_1009 = low_1009 + (high_1009 - low_1009) / 2;
15
16         // sort low, mid, high element
17         if (arr_1009[low_1009] > arr_1009[mid_1009]) {
18             swap_1009(arr_1009, low_1009, mid_1009);
19         }
20         if (arr_1009[low_1009] > arr_1009[high_1009]) {
21             swap_1009(arr_1009, low_1009, high_1009);
22         }
23         if (arr_1009[mid_1009] > arr_1009[high_1009]) {
24             swap_1009(arr_1009, mid_1009, high_1009);
25         }
26         swap_1009(arr_1009, mid_1009, high_1009);
27     }
28
29     // func : using median of three
30     static int partition_1009(int[] arr_1009, int low_1009, int high_1009) {
31         // calls medianOfThree method before initialising a pivot
32         medianOfThree(arr_1009, low_1009, high_1009);
33
34         int pivot_1009 = arr_1009[high_1009];
35         int i_1009 = (low_1009 - 1);
36
37         for (int j_1009 = low_1009; j_1009 <= high_1009 - 1; j_1009++) {
38             // if the current element is smaller or equal than pivot
39             if (arr_1009[j_1009] < pivot_1009) {
40                 // increment of smaller index element
41                 i_1009++;
42                 swap_1009(arr_1009, i_1009, j_1009);
43             }
44         }
45         swap_1009(arr_1009, i_1009 + 1, high_1009);
46         return (i_1009 + 1);
47     }
48 }
```



```

50● static void quickSort_1009(int[] arr_1009, int low_1009, int high_1009) {
51●     if (low_1009 < high_1009) {
52         int pi_1009 = partition_1009(arr_1009, low_1009, high_1009);
53         quickSort_1009(arr_1009, low_1009, pi_1009 - 1);
54         quickSort_1009(arr_1009, pi_1009 + 1, high_1009);
55     }
56 }
57
58● public static void printArr_1009(int[] arr_1009) {
59●     for (int i_1009 = 0; i_1009 < arr_1009.length; i_1009++) {
60         System.out.print(arr_1009[i_1009] + " ");
61     }
62     System.out.println();
63 }

```

```

<terminated> QuickSort_2511531009 [Java Application] C:\Users\han
Data sebelum diurutkan : 10 7 8 9 1 5
Data Terurut quick sort: 1 5 7 8 9 10

```

Shell Sort Memiliki range kompleksitas waktu sebagai berikut :

Aspek	Kompleksitas	Keterangan
<i>Best Case</i>	$O(n \log n)$	Ketika partisi seimbang
<i>Average Case</i>	$O(n \log n)$	Berdasarkan kebanyakan kasus.
<i>Worst Case</i>	$O(n^2)$	Terjadi ketika algoritma telah berurut atau dibalik.

Kompleksitas ruang yang dimiliki oleh algoritma ini adalah $O(\log n)$ karena terdapat program yang rekursif

Dengan menggunakan algoritma Shell Sort terdapat beberapa keuntungan.

Keuntungan dari menggunakan Shell Sort ialah :

- Cepat untuk dataset yang besar
- Algoritma pengurutan ditempatkan
- Efisien untuk data random atau besar

Walaupun demikian, kekurangan Shell Sort ialah :

- Tidak cukup stabil
- Kompleksitas waktu $O(n^2)$
- Dapat menyebabkan stack overflow

Dengan mengetahui keuntungan dan kekurangan, kita dapat memahami kapan untuk menggunakan algoritma Shell Sort :

- Ketika kita mengurutkan dataset yang besar
- Ketika performa rata-rata lebih penting daripada *worst-case*

- Ketika ingin mengurutkan secara ditempat

2.3 MergeSort

Merge Sort merupakan algoritma pengurutan yang juga bekerja berdasarkan strategi divide and conquer. Array dibagi menjadi dua bagian yang sama besar secara rekursif hingga setiap bagian hanya memiliki satu elemen, yang secara otomatis dianggap sudah terurut.

Setelah pembagian selesai, proses penggabungan (merge) dilakukan dengan membandingkan elemen-elemen dari dua sub-array secara berpasangan. Elemen yang lebih kecil diambil terlebih dahulu dan ditempatkan ke dalam array hasil, hingga seluruh elemen dari kedua sub-array telah digabungkan dalam urutan yang benar. Proses ini berulang secara rekursif hingga seluruh array kembali tergabung menjadi satu.

```

1 package pekan8_2511531009;
2
3 public class MergeSort_2511531009 {
4
5     void merge_1009(int arr_1009[], int l_1009, int m_1009, int r_1009) {
6         // find sizes of
7         int n1_1009 = m_1009 - l_1009 + 1;
8         int n2_1009 = r_1009 - m_1009;
9
10        // temp arrays
11        int L_1009[] = new int[n1_1009];
12        int R_1009[] = new int[n2_1009];
13
14        // copy data to temp
15        for (int i_1009 = 0; i_1009 < n1_1009; ++i_1009) {
16            L_1009[i_1009] = arr_1009[l_1009 + i_1009];
17        }
18
19        for (int j_1009 = 0; j_1009 < n2_1009; ++j_1009) {
20            R_1009[j_1009] = arr_1009[m_1009 + 1 + j_1009]; // 1 -> m_1009 + 1
21        }
22
23        int i_1009 = 0, j_1009 = 0;
24        // initial index of merged subarray array
25        int k_1009 = l_1009;
26        while (i_1009 < n1_1009 && j_1009 < n2_1009) {
27            if (L_1009[i_1009] <= R_1009[j_1009]) {
28                arr_1009[k_1009] = L_1009[i_1009];
29                i_1009++;
30            } else {
31                arr_1009[k_1009] = R_1009[j_1009];
32                j_1009++;
33            }
34            k_1009++;
35        }

```

```

37 //copy remaining element of L[] if any
38 while (i_1009 < n1_1009) {
39     arr_1009[k_1009] = L_1009[i_1009];
40     i_1009++;
41     k_1009++;
42 }
43
44 //copy remaining element of R[] if any
45 while (j_1009 < n2_1009) {
46     arr_1009[k_1009] = R_1009[j_1009];
47     j_1009++;
48     k_1009++;
49 }
50 }
51
52 void sort_1009(int arr[], int l_1009, int r_1009) {
53     if (l_1009 < r_1009) {
54         // find the mid point
55         int m_1009 = (l_1009 + r_1009) / 2;
56
57         // sort first and second halves
58         sort_1009(arr, l_1009, m_1009);
59         sort_1009(arr, m_1009 + 1, r_1009); // m_1009 + l_1009 -> m_1009 + 1
60
61         // merge the sorted halves
62         merge_1009(arr, l_1009, m_1009, r_1009);
63     }
64 }

```

```

66 // a utility funct to print arr of size n
67 static void printArray_1009(int arr_1009[]) {
68     int n_1009 = arr_1009.length;
69     for (int i_1009 = 0; i_1009 < n_1009; i_1009++) {
70         System.out.print(arr_1009[i_1009] + " ");
71     }
72     System.out.println();
73 }

```

```

75 public static void main(String[] args) {
76     int arr_1009[] = {12, 11, 13, 5, 6, 7};
77     System.out.println("Sebelum terurut : ");
78     printArray_1009(arr_1009);
79
80     MergeSort_2511531009 ob_1009 = new MergeSort_2511531009();
81
82     ob_1009.sort_1009(arr_1009, 0, arr_1009.length - 1);
83     System.out.println("\nSesudah Terurut menggunakan Merge Sort :");
84     printArray_1009(arr_1009);
85 }
86 }

```

```

<terminated> MergeSort_2511531009 [Java Application] C:\Users\hamdi\p2\poc
Sebelum terurut :
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort :
5 6 7 11 12 13

```

Shell Sort Memiliki range kompleksitas waktu sebagai berikut :

Aspek	Kompleksitas	Keterangan
<i>Best Case</i>	$O(n \log n)$	Ketika partisi seimbang
<i>Average Case</i>	$O(n \log n)$	Berdasarkan kebanyakan kasus.

<i>Worst Case</i>	$O(n^2)$	Terjadi ketika algoritma telah berurutan atau dibalik.
-------------------	----------	--

Kompleksitas ruang yang dimiliki oleh algoritma ini adalah $O(\log n)$ karena terdapat program yang rekursif

Dengan menggunakan algoritma Shell Sort terdapat beberapa keuntungan.

Keuntungan dari menggunakan Shell Sort ialah :

- Cepat untuk dataset yang besar
- Algoritma pengurutan ditempatkan
- Efisien untuk data random atau besar

Walaupun demikian, kekurangan Shell Sort ialah :

- Tidak cukup stabil
- Kompleksitas waktu $O(n^2)$
- Dapat menyebabkan stack overflow

Dengan mengetahui keuntungan dan kekurangan, kita dapat memahami kapan untuk menggunakan algoritma Shell Sort :

- Ketika kita mengurutkan dataset yang besar
- Ketika performa rata-rata lebih penting daripada *worst-case*
- Ketika ingin mengurutkan secara ditempatkan

2.4 BubbleSortGUI

```

1 package pekan8_2511531009;
2
3 import javax.swing.*;
4
5 @SuppressWarnings("serial")
6 public class BubbleSortGUI_2511531009 extends JFrame {
7
8     private JTextField inputField_1009;
9     private JButton setButton_1009;
10    private JButton stepButton_1009;
11    private JButton resetButton_1009;
12    private JTextArea stepArea_1009;
13    private JPanel panelArray_1009;
14
15    private int[] array_1009;
16    private JLabel[] labelArray_1009;
17
18    private int i_1009 = 0;
19    private int j_1009 = 0;
20    private int stepCount_1009 = 1;
21    private boolean sorting_1009 = false;
22
23 }

```

```

26● public BubbleSortGUI_2511531009() {
27     setTitle("Visualisasi Bubble Sort");
28     setSize(800, 500);
29     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
30     setLocationRelativeTo(null);
31     setLayout(new BorderLayout(10, 10));
32
33     // Panel input bagian atas
34     JPanel inputPanel = new JPanel(new FlowLayout());
35
36     JLabel inputLabel = new JLabel("Masukkan angka (pisahkan dengan koma):");
37     JTextField inputField_1009 = new JTextField(30);
38
39     JButton setButton_1009 = new JButton("Set Array");
40     JButton stepButton_1009 = new JButton("Step");
41     JButton resetButton_1009 = new JButton("Reset");
42
43     stepButton_1009.setEnabled(false);
44
45     inputPanel.add(inputLabel);
46     inputPanel.add(inputField_1009);
47     inputPanel.add(setButton_1009);
48     inputPanel.add(stepButton_1009);
49     inputPanel.add(resetButton_1009);
50
51     add(inputPanel, BorderLayout.NORTH);
52
53     // Panel untuk menampilkan array
54     JPanel panelArray_1009 = new JPanel(new FlowLayout());
55     panelArray_1009.setPreferredSize(new Dimension(750, 120));
56     add(panelArray_1009, BorderLayout.CENTER);
57
58     // Text area untuk menampilkan langkah sorting
59     JTextArea stepArea_1009 = new JTextArea();
60     stepArea_1009.setEditable(false);
61     stepArea_1009.setFont(new Font("Monospaced", Font.PLAIN, 14));
62
63     JScrollPane scrollPane = new JScrollPane(stepArea_1009);
64     scrollPane.setPreferredSize(new Dimension(750, 250));
65
66     add(scrollPane, BorderLayout.SOUTH);
67
68     // Event tombol
69●   setButton_1009.addActionListener(new ActionListener() {
70       @Override
71●       public void actionPerformed(ActionEvent e) {
72           setArrayFromInput();
73       }
74   });
75
76●   stepButton_1009.addActionListener(new ActionListener() {
77       @Override
78●       public void actionPerformed(ActionEvent e) {
79           performStep();
80       }
81   });
82
83●   resetButton_1009.addActionListener(new ActionListener() {
84       @Override
85●       public void actionPerformed(ActionEvent e) {
86           reset();
87       }
88   });
89 }

```

```

91● private void setArrayFromInput() {
92     String text = inputField_1009.getText().trim();
93
94●     if (text.isEmpty()) {
95         return;
96     }
97
98     String[] parts_1009 = text.split(",");
99     array_1009 = new int[parts_1009.length];
100
101●     try {
102●         for (int k = 0; k < parts_1009.length; k++) {
103             array_1009[k] = Integer.parseInt(parts_1009[k].trim());
104         }
105●     } catch (NumberFormatException e) {
106         JOptionPane.showMessageDialog(
107             this,
108             "Masukkan hanya angka yang dipisahkan koma!",
109             "Error",
110             JOptionPane.ERROR_MESSAGE
111         );
112         return;
113     }
114
115     i_1009 = 0;
116     j_1009 = 0;
117     stepCount_1009 = 1;
118     sorting_1009 = true;
119
120     stepButton_1009.setEnabled(true);
121     stepArea_1009.setText("");
122     panelArray_1009.removeAll();
123
124     labelArray_1009 = new JLabel[array_1009.length];

```



```

126●     for (int k_1009 = 0; k_1009 < array_1009.length; k_1009++) {
127         labelArray_1009[k_1009] = new JLabel(String.valueOf(array_1009[k_1009]));
128         labelArray_1009[k_1009].setFont(new Font("Arial", Font.BOLD, 24));
129         labelArray_1009[k_1009].setOpaque(true);
130         labelArray_1009[k_1009].setBackground(Color.WHITE);
131         labelArray_1009[k_1009].setBorder(BorderFactory.createLineBorder(Color.
132             labelArray_1009[k_1009].setPreferredSize(new Dimension(50, 50));
133             labelArray_1009[k_1009].setHorizontalAlignment(SwingConstants.CENTER);
134
135         panelArray_1009.add(labelArray_1009[k_1009]);
136     }
137
138     panelArray_1009.revalidate();
139     panelArray_1009.repaint();
140 }

```

```

142● private void performStep_1009() {
143●     if (!sorting_1009 || i_1009 >= array_1009.length - 1) {
144         sorting_1009 = false;
145         stepButton_1009.setEnabled(false);
146         JOptionPane.showMessageDialog(this, "Sorting selesai!");
147         return;
148     }
149
150     resetHighlights_1009();
151
152     StringBuilder stepLog_1009 = new StringBuilder();
153
154     labelArray_1009[j_1009].setBackground(Color.CYAN);
155     labelArray_1009[j_1009 + 1].setBackground(Color.CYAN);
156
157●     if (array_1009[j_1009] > array_1009[j_1009 + 1]) {
158         // Swap
159         int temp_1009 = array_1009[j_1009];
160         array_1009[j_1009] = array_1009[j_1009 + 1];
161         array_1009[j_1009 + 1] = temp_1009;
162
163         labelArray_1009[j_1009].setBackground(Color.RED);
164         labelArray_1009[j_1009 + 1].setBackground(Color.RED);
165
166         stepLog_1009.append("Langkah ").append(stepCount_1009).append(": ")
167             .append("Menukar elemen ke-").append(j_1009)
168             .append(" (").append(array_1009[j_1009 + 1]).append(")")
169             .append(" dengan ke-").append(j_1009 + 1)
170             .append(" (").append(array_1009[j_1009]).append(")\n");
171
172●     } else {
173         stepLog_1009.append("Langkah ").append(stepCount_1009).append(": ")
174             .append("Tidak ada pertukaran antara ke-")
175             .append(j_1009).append(" dan ke-")
176             .append(j_1009 + 1).append(")\n");
177     }

```

```

179     stepLog_1009.append("Hasil: ").append(arrayToString_1009(array_1009)).append("\n");
180     stepArea_1009.append(stepLog_1009.toString());
181
182     updateLabels();
183
184     j_1009++;
185
186●     if (j_1009 >= array_1009.length - 1 - i_1009) {
187         j_1009 = 0;
188         i_1009++;
189     }
190
191     stepCount_1009++;
192
193●     if (i_1009 >= array_1009.length - 1) {
194         sorting_1009 = false;
195         stepButton_1009.setEnabled(false);
196         resetHighlights_1009();
197
198●         for (JLabel label_1009 : labelArray_1009) {
199             label_1009.setBackground(Color.GREEN);
200         }
201
202         JOptionPane.showMessageDialog(this, "Sorting selesai!");
203     }
204 }

```

```

206● private void updateLabels_1009() {
207●     for (int k_1009 = 0; k_1009 < array_1009.length; k_1009++) {
208         labelArray_1009[k_1009].setText(String.valueOf(array_1009[k_1009]));
209     }
210 }
211
212● private void resetHighlights_1009() {
213●     for (JLabel label_1009 : labelArray_1009) {
214         label_1009.setBackground(Color.WHITE);
215     }
216 }
217
218● private void reset_1009() {
219     inputField_1009.setText("");
220     panelArray_1009.removeAll();
221     panelArray_1009.revalidate();
222     panelArray_1009.repaint();
223
224     stepArea_1009.setText("");
225     stepButton_1009.setEnabled(false);
226
227     sorting_1009 = false;
228     i_1009 = 0;
229     j_1009 = 0;
230     stepCount_1009 = 1;
231 }
232
233● private String arrayToString_1009(int[] arr) {
234     StringBuilder sb_1009 = new StringBuilder();
235
236●     for (int k_1009 = 0; k_1009 < arr.length; k_1009++) {
237         sb_1009.append(arr[k_1009]);
238
239●         if (k_1009 < arr.length - 1) {
240             sb_1009.append(", ");
241         }
242     }
243
244     return sb_1009.toString();
245 }

```

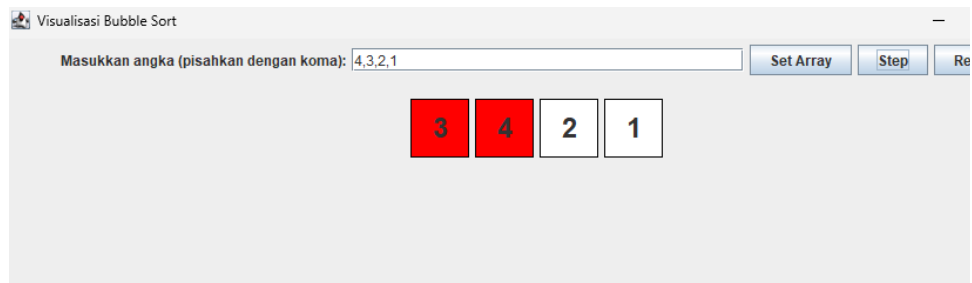
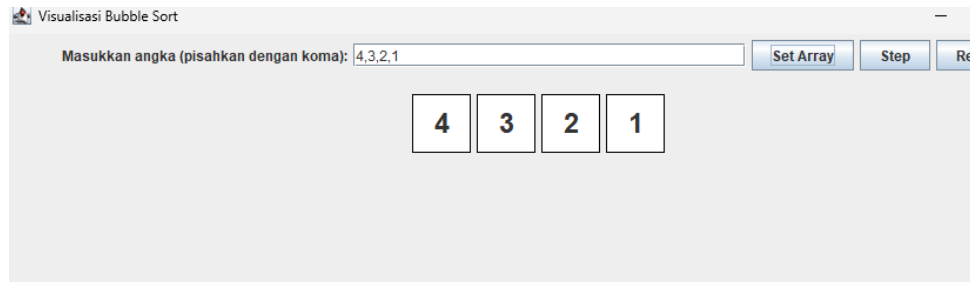
```

247● public static void main(String[] args) {
248●     SwingUtilities.invokeLater(new Runnable() {
249         @Override
250●         public void run() {
251             new BubbleSortGUI_2511531009().setVisible(true);
252         }
253     });
254 }
255 }

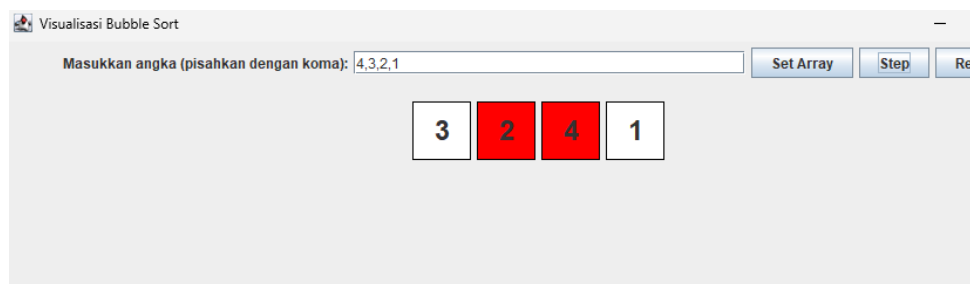
```

Visualisasi Bubble Sort

Masukkan angka (pisahkan dengan koma):

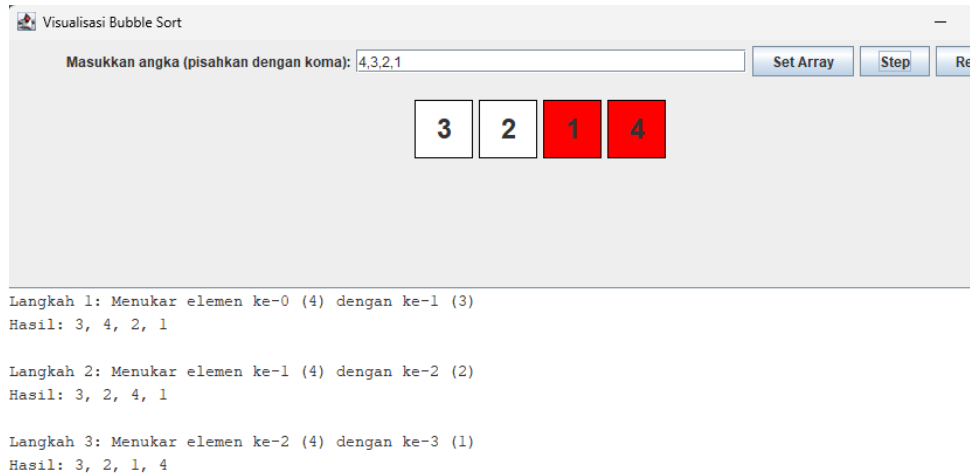


Langkah 1: Menukar elemen ke-0 (4) dengan ke-1 (3)
Hasil: 3, 4, 2, 1

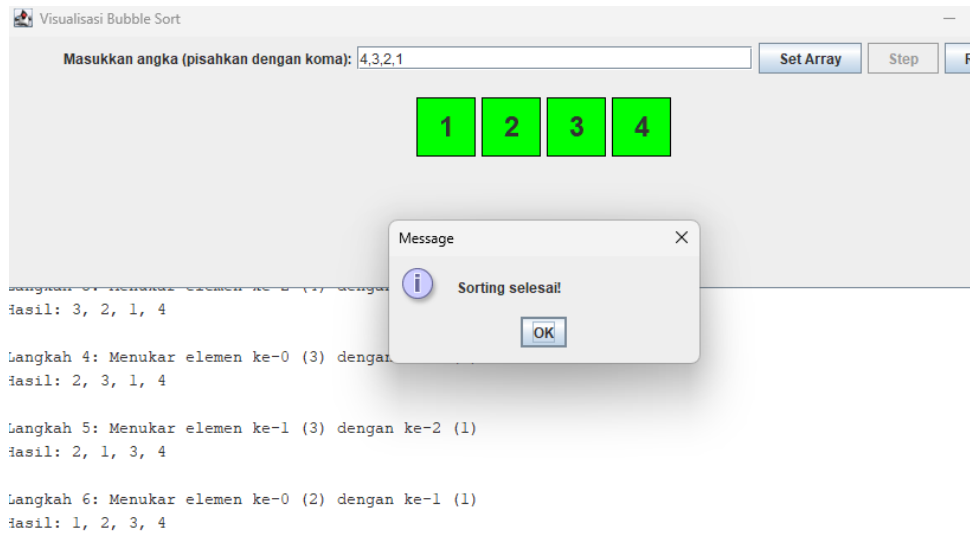


Langkah 1: Menukar elemen ke-0 (4) dengan ke-1 (3)
Hasil: 3, 4, 2, 1

Langkah 2: Menukar elemen ke-1 (4) dengan ke-2 (2)
Hasil: 3, 2, 4, 1



Kemudian langkah berlanjut hingga semua element pada array telah terurut



BAB III KESIMPULAN

1.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Sorting memudahkan kita untuk membaca data dengan terurut sesuai yang kita inginkan.

1.2 Saran

Seperti biasa, mahasiswa sebaiknya dapat diarahkan dengan lebih baik selama praktikum berlangsung dengan metode pembelajaran yang lebih engaging dan interaktif, untuk menghindari suasana yang monoton dan canggung.

BAB IV

DAFTAR PUSTAKA

- [1] *Shellsort* [Daring]. Tersedia: <https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/Shellsort.html>. Diakses: 30 Mei 2026.
- [2] *Shell Sort* [Daring]. Tersedia: <https://www.algorithmroom.com/dsa/shell-sort>. Diakses: 30 Mei 2026.
- [3] *Quick Sort* [Daring]. Tersedia: <https://www.algorithmroom.com/dsa/quick-sort>. Diakses: 30 Mei 2026.
- [4] *Merge Sort* [Daring]. Tersedia: <https://www.algorithmroom.com/dsa/merge-sort>. Diakses: 30 Mei 2026.

LAPORAN TUGAS PEKAN 8 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2026

Lampiran Tugas

Topik : Algoritma Sorting (Shell, Quick, dan Merge)

Tema : Mengurutkan Playlist Lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

2. Aturan Penamaan

- a) Nama Kelas : concat sufiks “_” + NIM
Contoh : sorting_2511531009
- b) Nama Variabel dan Method : concat sufiks “_” + NIM mod 10^4
Contoh : dataLagu_1009
shellSort_1009
- c) Method Utama
Contoh : quickSort_1009 if choose == quick-sort
shellSort_1009 if choose == shell-sort
mergeSort_1009 if choose == merge-sort

3. Struktur Program

a. Kelas Lagu (lagu_2511531009)

Atribut :

- Judul Lagu := judul_1009 (String)
- Penyanyi Lagu := penyanyi_1009 (String)
- Durasi Lagu := durasi_1009 (Int)

Method :

- Constructor untuk menginisialisasi semua atribut
- Selektor (Getter) untuk semua atribut
- Mutator (Setter) untuk setiap atribut
- toString Override

b. Kelas Sorting (sorting_2511531009)

Minimal harus ada :

- Array dataLagu_1009 : menyimpan lagu (maks : 20)
- Method inputData_1009 : mengisi data awal (min : 7 lagu)
- Method pilihAlgoritma : XOR(
shellSort_1009; → Ascending String judul

```
quickSort_1009; → Ascending durasi
mergeSort      → Ascending String judul
)
```

- iv. Method tampilData_1009 : menampilkan sebelum dan sesudah sorting

4. Ketentuan Pilihan

Pilih hanya 1_algoritma, beserta alasan memilih algoritma tersebut (dengan panjang 4-5 kalimat). Tidak perlu membuat ketiganya, fokus pada implementasi yang benar dan sesuai aturan penamaan.

5. Contoh Output

=== Sorting Playlist NIM : 2511531009 ===

Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:

1. Mio Cristo Piange Diamanti – 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik

Data Setelah Quick Sort (Durasi Asc):

1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik

6. Kriteria Penilaian

1. Algoritma berjalan benar sesuai pilihan
2. Aturan penamaan NIM 100% dipatuhi
3. Sorting bisa untuk minimal 7 data
4. Menampilkan data sebelum dan sesudah
5. Laporan menjelaskan pilihan algoritma dan kompleksitasnya

7. Deadline

Tugas dikumpulkan paling lambat pada hari Senin, tanggal 1 Juni 2026, pukul 23.59 WIB.

Java (ADT) Class · lagu_2511531009

Kelas ini menginisialisasikan atribut yang akan digunakan untuk digunakan pada class driver.

```
1 package pekan8_2511531009;
2
3 public class lagu_2511531009 {
4     // Atribut
5     private String judul_1009;
6     private String penyanyi_1009;
7     private int durasi_1009;
8
9     // Constructor
10    public lagu_2511531009(String judul_1009, String penyanyi_1009, int durasi_1009) {
11        this.judul_1009 = judul_1009;
12        this.penyanyi_1009 = penyanyi_1009;
13        this.durasi_1009 = durasi_1009;
14    }
15
16    // Getter
17    public String getJudul_1009() { return judul_1009; }
18    public String getPenyanyi_1009() { return penyanyi_1009; }
19    public int getDurasi_1009() { return durasi_1009; }
20
21    // Setter
22    public void setJudul_1009(String judul_1009) { this.judul_1009 = judul_1009; }
23    public void setPenyanyi_1009(String penyanyi_1009) { this.penyanyi_1009 = penyanyi_1009; }
24    public void setDurasi_1009(int durasi_1009) { this.durasi_1009 = durasi_1009; }
25
26    // toString Override
27    @Override
28    public String toString() {
29        return judul_1009 + " - " + penyanyi_1009 + " - " + durasi_1009 + " detik";
30    }
31 }
```

Java (Main/Driver) Class · sorting_2511531009

```
1 package pekan8_2511531009;
2 import java.util.Scanner;
3
4 public class sorting_2511531009 {
5
6     private lagu_2511531009[] dataLagu_1009;
7     private int jumlahLagu_1009;
8
9     public sorting_2511531009() {
10         dataLagu_1009 = new lagu_2511531009[20];
11         jumlahLagu_1009 = 0;
12     }
13
14     public void inputData_1009(Scanner scanner_1009) {
15         while (true) {
16             try {
17                 System.out.print("Masukkan jumlah lagu (min 7, maks 20): ");
18                 jumlahLagu_1009 = Integer.parseInt(scanner_1009.nextLine());
19
20                 if (jumlahLagu_1009 >= 7 && jumlahLagu_1009 <= 20) {
21                     break;
22                 } else {
23                     System.out.println("[Error] Jumlah lagu harus di antara 7 dan 20");
24                 }
25             } catch (NumberFormatException e) {
26                 System.out.println("[Error] Input tidak valid! Harap masukkan angka");
27             }
28         }
29
30         System.out.println("\nMasukkan data lagu:");
31         for (int i = 0; i < jumlahLagu_1009; i++) {
32             System.out.println("Lagu ke-" + (i + 1) + ":");
33
34             String judul = "";
35             while (true) {
36                 System.out.print("Judul : ");
37                 judul = scanner_1009.nextLine().trim();
38                 if (!judul.isEmpty()) {
39                     break;
40                 }
41                 System.out.println("[Error] Judul lagu tidak boleh kosong!");
42             }
43
44             String penyanyi = "";
45             while (true) {
46                 System.out.print("Penyanyi : ");
47                 penyanyi = scanner_1009.nextLine().trim();
48                 if (!penyanyi.isEmpty()) {
49                     break;
50                 }
51                 System.out.println("[Error] Nama penyanyi tidak boleh kosong!");
52             }
53
54             int durasi = 0;
55             while (true) {
56                 try {
57                     System.out.print("Durasi (detik) : ");
58                     durasi = Integer.parseInt(scanner_1009.nextLine());
59                     if (durasi > 0) {
60                         break;
61                     }
62                     System.out.println("[Error] Durasi harus lebih besar dari 0 detik");
63                 } catch (NumberFormatException e) {
64                     System.out.println("[Error] Input tidak valid! Harap masukkan angka");
65                 }
66             }
67
68             dataLagu_1009[i] = new lagu_2511531009(judul, penyanyi, durasi);
69         }
70     }
71 }
```

```

72● public void shellSort_1009() {
73     int n = jumlahLagu_1009;
74
75●     for (int gap = n / 2; gap > 0; gap /= 2) {
76●         for (int i = gap; i < n; i++) {
77             lagu_2511531009 temp = dataLagu_1009[i];
78             int j;
79
80●             for (j = i; j >= gap && dataLagu_1009[j - gap].getJudul_1009()
81                 .compareToIgnoreCase(temp.getJudul_1009()) > 0; j -= gap) {
82                 dataLagu_1009[j] = dataLagu_1009[j - gap];
83             }
84
85             dataLagu_1009[j] = temp;
86         }
87     }
88 }
89
90● public void tampilData_1009() {
91     System.out.println();
92     System.out.println("Data Sebelum Sorting:");
93●     for (int i = 0; i < jumlahLagu_1009; i++) {
94         System.out.println("\n" + dataLagu_1009[i].getJudul_1009()
95             + " - " + dataLagu_1009[i].getDurasi_1009() + " detik");
96     }
97
98     shellSort_1009();
99
100     System.out.println();
101     System.out.println("Data Setelah Shell Sort (Judul Asc):");
102●     for (int i = 0; i < jumlahLagu_1009; i++) {
103         System.out.println((i + 1) + ". " + dataLagu_1009[i].getJudul_1009()
104             + " - " + dataLagu_1009[i].getDurasi_1009() + " detik");
105     }
106 }
107
108● public static void main(String[] args) {
109     Scanner scanner = new Scanner(System.in);
110
111     System.out.println("=== Sorting Playlist NIM : 2511531009 ===");
112     System.out.println("Algoritma : Shell Sort (Ascending Judul)");
113
114     sorting_2511531009 playlist = new sorting_2511531009();
115     playlist.inputData_1009(scanner);
116     playlist.tampilData_1009();
117
118     scanner.close();
119 }
120 }

```

```

=== Sorting Playlist NIM : 2511531009 ===
Algoritma : Shell Sort (Ascending Judul)
Masukkan jumlah lagu (min 7, maks 20): sfs
[Error] Input tidak valid! Harap masukkan angka bulat.
Masukkan jumlah lagu (min 7, maks 20): 7

Masukkan data lagu:
Lagu ke-1:
Judul   : world.execute(me);
Penyanyi : Mili
Durasi (detik) : 212
Lagu ke-2:
Judul   : Duvet
Penyanyi : Boa
Durasi (detik) : 203
Lagu ke-3:
Judul   : Peradaban
Penyanyi : .Feast
Durasi (detik) : 339
Lagu ke-4:
Judul   : This Charming Man
Penyanyi : The Smiths
Durasi (detik) : 163
Lagu ke-5:
Judul   : Linger
Penyanyi : The Cranberries
Durasi (detik) : 274
Lagu ke-6:
Judul   : In Hell We Live, Lament
Penyanyi : Mili
Durasi (detik) : 215
Lagu ke-7:
Judul   : There is a Light That Never Goes Out
Penyanyi : The Smiths
Durasi (detik) : 243
Data Sebelum Sorting:

world.execute(me); - 212 detik

Duvet - 203 detik

Peradaban - 339 detik

This Charming Man - 163 detik

Linger - 274 detik

In Hell We Live, Lament - 215 detik

There is a Light That Never Goes Out - 243 detik

Data Setelah Shell Sort (Judul Asc):
1. Duvet - 203 detik
2. In Hell We Live, Lament - 215 detik
3. Linger - 274 detik
4. Peradaban - 339 detik
5. There is a Light That Never Goes Out - 243 detik
6. This Charming Man - 163 detik
7. world.execute(me); - 212 detik

```

Shell Sort dipilih karena merupakan algoritma pengurutan (sorting) yang efisien untuk dataset berukuran sedang, seperti playlist lagu yang jumlahnya terbatas (maksimal 20 lagu). Algoritma ini memiliki kompleksitas waktu yang lebih baik daripada Bubble Sort atau Insertion Sort konvensional. Oleh karena itu, Shell Sort sangat cocok untuk mengurutkan data berdasarkan judul lagu secara ascending (menaik). Selain itu, Shell Sort merupakan algoritma yang tidak memerlukan memori tambahan yang besar,

sehingga efisien dalam penggunaan ruang memori. Meskipun secara teoretis termasuk unstable sort, algoritma ini memberikan performa yang konsisten untuk data berukuran kecil hingga menengah, dengan implementasi yang relatif lebih sederhana daripada Quick Sort atau Merge Sort.

Serta Penulis secara pribadi telah menambahkan error management untuk mencegah terjadinya perulangan pemasukan penginputan data dari awal yang cukup menjengkelkan.